

AgroVision

Documento de Arquitetura e Attention Leakage (AL)

Arquitetura do sistema, pipeline XAI e métricas originais **Attention Leakage (AL)** e **Attention Focus Score (AFS)**.

Autor: Luiz Carlos

Trabalho de Conclusão de Curso — TCC · 2026

1. Visão Geral do Sistema

O AgroVision é um aplicativo mobile *offline-first* para identificação de espécies vegetais. A classificação ocorre integralmente no dispositivo, usando um modelo TFLite embarcado. Quando o pesquisador precisa inspecionar o comportamento espacial do modelo, um **Modo Científico** opt-in consulta um backend Python (FastAPI) que executa Grad-CAM e devolve métricas de atenção.

O sistema é composto por três pilares independentes que se articulam em tempo de execução:

- **Aplicativo Mobile** — React Native + Expo. Inferência offline via TFLite, histórico SQLite, UI nativa. Contém o cliente `xaiService.ts` que aciona o modo científico.
- **Backend XAI** — FastAPI + Python, rodando no Railway. Recebe a imagem via HTTP multipart, executa Grad-CAM sobre o modelo Keras, computa as métricas AL/AFS e devolve heatmap e overlay em base64.
- **Pipeline de Treinamento** — scripts Python que produzem dois artefatos: `model_species.h5` (para o backend) e `model_species.tflite` (para o app).

Decisão arquitetural central: o TFLite é um runtime de inferência puro — não expõe gradientes. Grad-CAM exige gradientes. Por isso o modelo Keras original é mantido como artefato separado e servido pelo backend.

2. Arquitetura dos Três Pilares

Arquitetura do Sistema AgroVision — Três Pilares

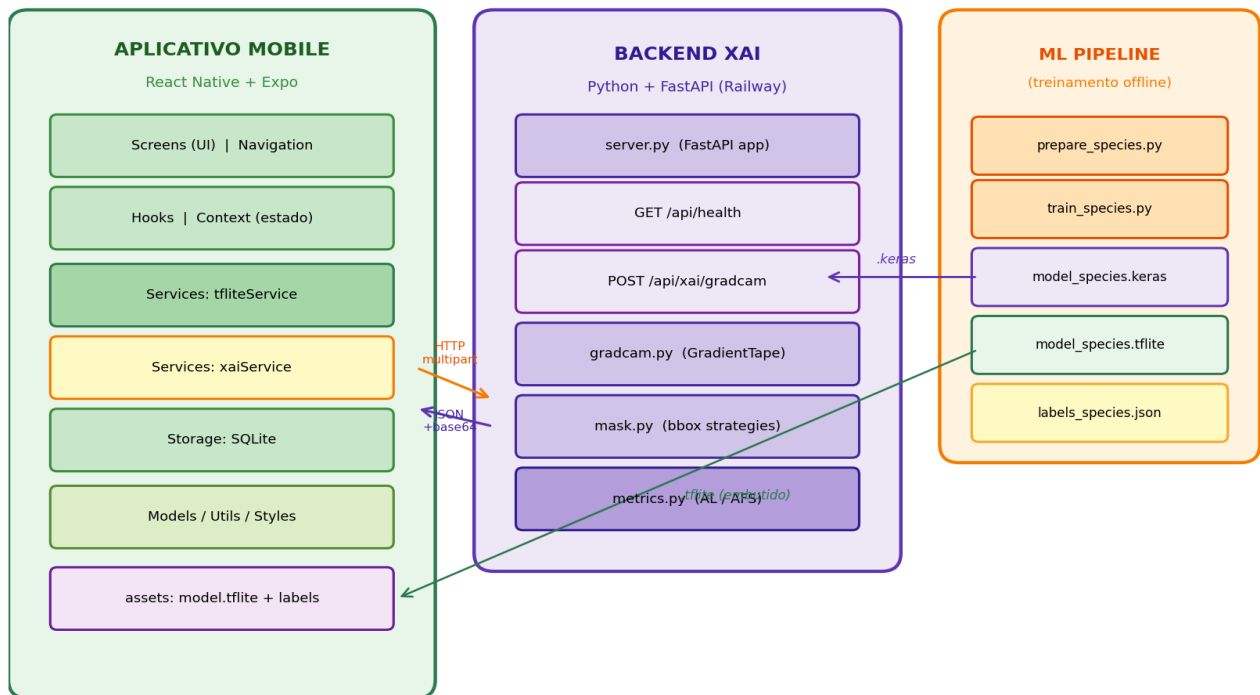


Figura 1. Visão macro do sistema. As setas laranja indicam artefatos produzidos pelo pipeline de treinamento; as setas verde/roxo indicam o fluxo HTTP entre app e backend durante o Modo Científico.

O backend é stateless por design: o modelo Keras é carregado uma única vez na inicialização (singleton ModelRegistry) e reutilizado em todas as requisições subsequentes. Isso elimina latência de I/O por requisição, relevante dado o tempo de carregamento do modelo (~2 s).

3. Arquitetura Interna do Aplicativo Mobile

Arquitetura Interna do Aplicativo Mobile — Camadas

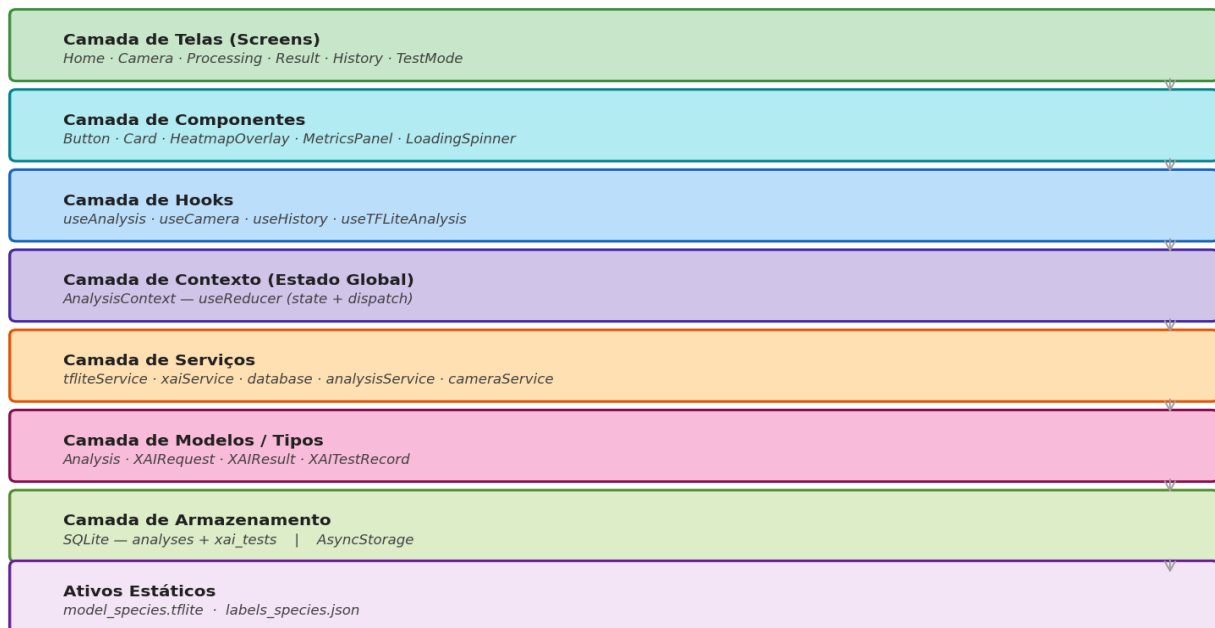


Figura 2. Camadas internas do aplicativo. A dependência flui de cima para baixo: Screens dependem de Hooks/Contexto, que dependem de Services, que dependem de Models e Storage.

3.1. Camadas e responsabilidades

- **Screens:** seis telas principais (Home, Camera, Processing, Result, History) mais TestModeScreen — exclusiva do Modo Científico. Cada tela é responsável apenas pela composição visual e pelo despacho de ações ao Contexto.
- **Componentes XAI:** HeatmapOverlay (renderiza o PNG base64 com toggle original/heatmap) e MetricsPanel (exibe AL, AFS, confiança, estratégia de bbox e layer usada com barra de progresso colorida).
- **Hooks:** encapsulam lógica reutilizável. useTFLiteAnalysis orquestra o pipeline offline. A TestModeScreen gerencia seu próprio estado local (runs, labels, export) sem passar pelo Contexto global.
- **AnalysisContext:** estado global via useReducer. Contém currentAnalysis, history, isLoading e error. O Modo Científico não usa este contexto — opera de forma isolada.
- **Services:** tfliteService (inferência offline, carrega pesos do bundle, redimensiona a 224×224 com jpeg-js, executa tf.predict no backend CPU) e xaiService (cliente Axios para o backend XAI, monta FormData com imagem + parâmetros, converte resposta snake_case → camelCase).
- **SQLite:** duas tabelas independentes. analyses armazena o histórico do fluxo normal. xai_tests armazena todos os campos do Modo Científico, incluindo o overlay em base64, para inspeção offline posterior e exportação CSV.

4. Arquitetura do Backend XAI

Dependências entre Módulos do Backend XAI

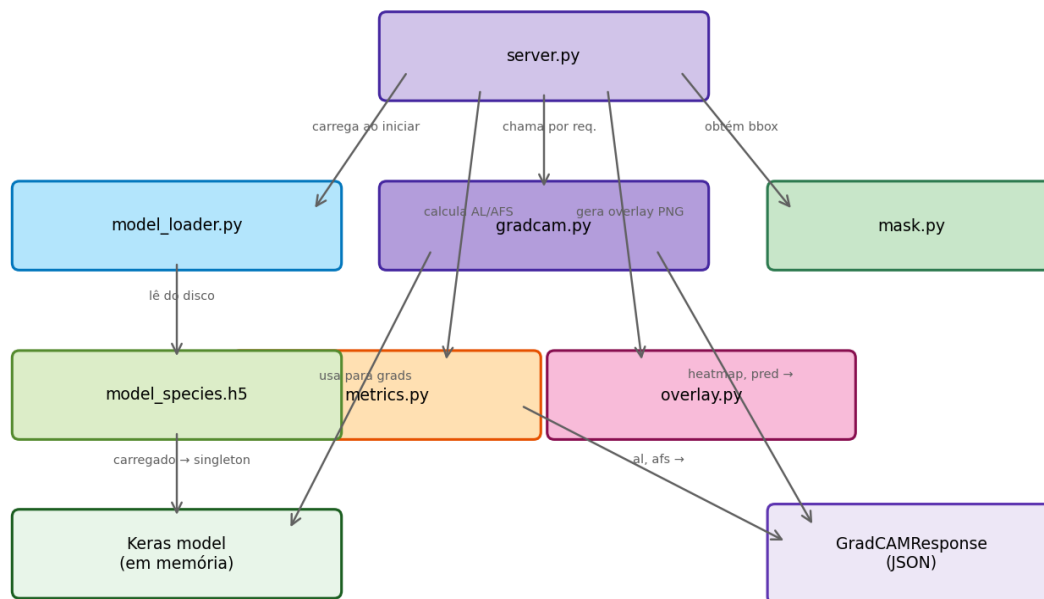


Figura 3. Grafo de dependências entre módulos do backend. O `server.py` é o ponto de entrada; os módulos XAI são *stateless* e recebem o modelo como parâmetro.

4.1. Endpoints

GET `/api/health` — chamado pelo app ao abrir a `TestModeScreen` (timeout 5 s). Retorna status, caminho do modelo, número de classes e as últimas camadas `Conv2D` disponíveis. Permite ao usuário confirmar conectividade e ver quais layers podem ser selecionadas.

POST `/api/xai/gradcam` — endpoint principal. Recebe `multipart/form-data` com a imagem e os parâmetros (`bbox_strategy`, `threshold`, `coverage`, `layer_name` opcional). Retorna o `GradCAMResponse` JSON com todos os campos de resultado. Timeout no cliente: 60 s.

4.2. Startup e singleton do modelo

Ao subir, o servidor executa `registry.load()`, que localiza o arquivo do modelo (prioriza `model_species.h5` sobre `.keras`), carrega via `tf.keras.models.load_model` com `compile=False` e indexa todas as camadas `Conv2D` — incluindo as aninhadas dentro do sub-modelo `MobileNetV2`. O modelo fica em memória para toda a vida útil do processo; cada requisição recebe uma referência ao mesmo objeto.

4.3. Módulo `gradcam.py`

Implementa `generate_gradcam`. O passo crítico é construir um modelo auxiliar (`_build_grad_model`) que, em uma única passagem *forward*, expõe tanto os feature maps da camada-alvo quanto a saída final. Isso é essencial: se os dois tensores fossem computados em passagens separadas, o `tf.GradientTape` não encontraria gradiente algum (retornaria `None`). Como o `MobileNetV2` é um sub-modelo aninhado, a função detecta este caso e reconstrói o grafo externo substituindo a

chamada ao sub-modelo por um modelo interno que expõe ambos os tensores.

5. Attention Leakage (AL) e Attention Focus Score (AFS)

AL e AFS são as métricas originais propostas neste trabalho. Elas respondem a uma pergunta que as métricas clássicas de classificação não respondem: *onde o modelo olhou* para tomar sua decisão?

5.1. Motivação

Acurácia, precisão e F1 descrevem o *quê* foi predito. São cegas à distribuição espacial da atenção do modelo. Dois modelos com acurácia idêntica podem ter comportamentos espaciais opostos: um pode olhar para a folha; outro, para um artefato de fundo correlacionado com a classe no dataset de treino. Em aplicações agrícolas de campo — onde iluminação, ângulo e fundo variam continuamente — um modelo que generaliza *espacialmente* é mais robusto.

5.2. Conceito e estrutura

O cálculo de AL/AFS envolve três elementos estruturais:

- **Heatmap Grad-CAM $H(x,y)$** — mapa 2D no intervalo $[0,1]$ que indica, para cada pixel, o quanto aquela região contribuiu para a decisão do modelo. Produzido por `gradcam.py` e redimensionado para a resolução original da imagem.
- **Máscara $M(x,y)$** — mapa binário que define a Região de Interesse (RoI): pixels dentro da planta valem 1, fora valem 0. Produzida por `mask.py` com uma das três estratégias (ver 5.4).
- **Threshold τ** — parâmetro configurável $[0,1]$ (padrão 0.5) que zera ativações abaixo do limiar antes do cálculo, reduzindo o impacto de ativações marginais e ruído de fundo do heatmap.

Com esses três elementos, `metrics.py` computa: **AL** = fração da atenção acima de τ que caiu *fora* da máscara M ; **AFS** = fração que caiu *dentro* de M . Como são complementares, $AL + AFS = 1$ sempre.

5.3. Pipeline estrutural de AL/AFS

Pipeline Completo do AL / AFS — Estrutura e Fluxo

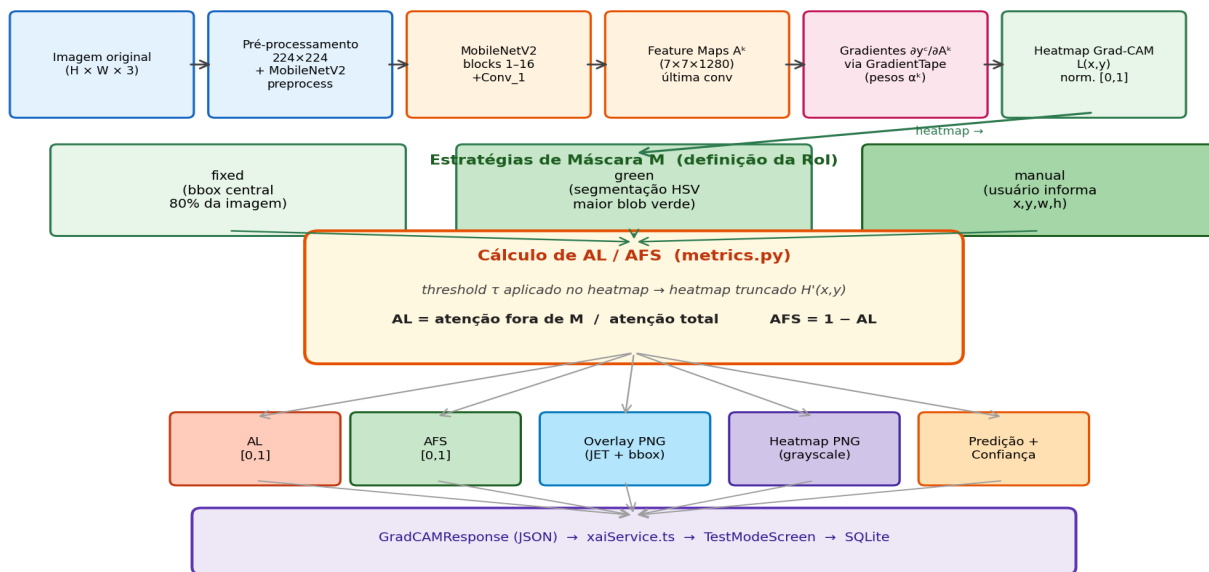


Figura 4. Fluxo completo de AL/AFS. O heatmap é produzido pelo Grad-CAM, a máscara pela estratégia de bbox e ambos confluem em metrics.py. O resultado é empacotado no GradCAMResponse e enviado ao app.

5.4. Estratégias de máscara e seu impacto em AL

A máscara determina o que se define como 'região correta'. A mesma imagem com heatmaps idênticos pode produzir valores de AL radicalmente diferentes dependendo do bbox usado. Três estratégias estão disponíveis:

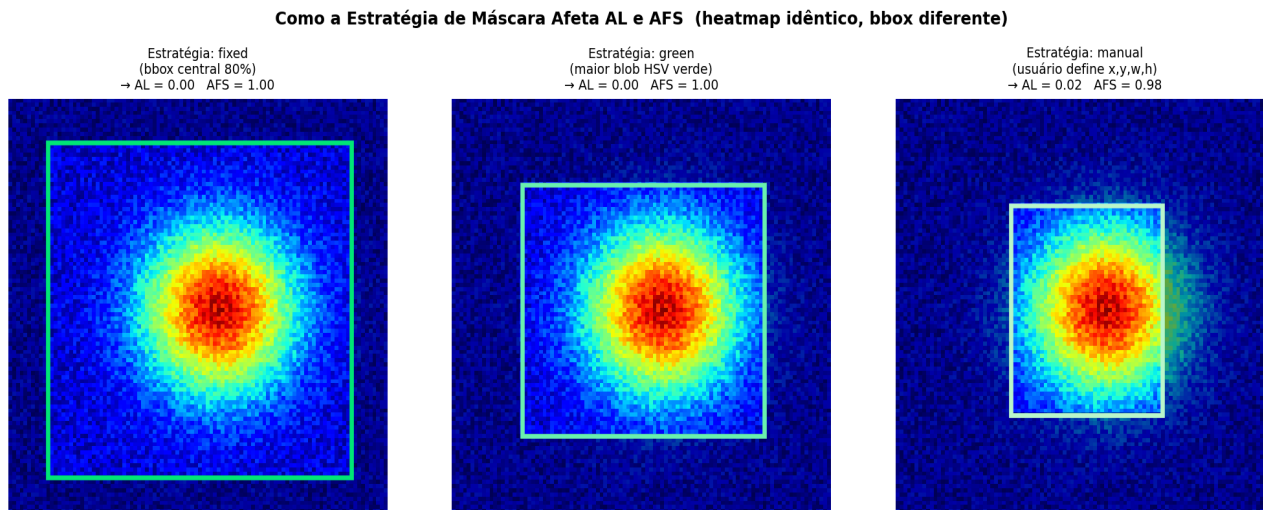


Figura 5. Mesmo heatmap Grad-CAM sob três estratégias de máscara. O bbox mais apertado (green/manual) captura melhor a planta real, tornando AL mais sensível a vazamentos finos.

- **fixed** — retângulo central de 80% da imagem. Rápido, determinístico, sem análise da imagem. Serve de baseline. Funciona bem para fotos enquadradas pelo usuário.
- **green** — segmentação HSV na faixa de verde (matiz 25–95), seguida de limpeza morfológica (abertura + fechamento) e extração do maior componente conexo. Se nenhum blob atinge 2% da área, o sistema recai para *fixed* automaticamente e reporta o fallback.
- **manual** — o usuário informa x,y,w,h diretamente. Permite análise sobre sub-regiões específicas (ex.: uma única folha dentro de uma planta maior).

5.5. Interpretação dos valores

Os valores de AL/AFS são exibidos no MetricsPanel com uma barra de progresso colorida (verde ≥ 0.7 · amarelo 0.4–0.7 · vermelho < 0.4):

AFS	AL	Interpretação	Ação sugerida
≥ 0.7	≤ 0.3	Atenção bem concentrada na planta	Resultado confiável; enquadramento adequado
0.4 – 0.7	0.3 – 0.6	Atenção parcialmente vazando	Verificar se há elementos de fundo próximos à planta
< 0.4	≥ 0.6	Modelo focando fora da planta	Imagem mal enquadrada ou viés no dataset; resultados susp

5.6. Fluxo de dados: da imagem ao AFS exibido no app

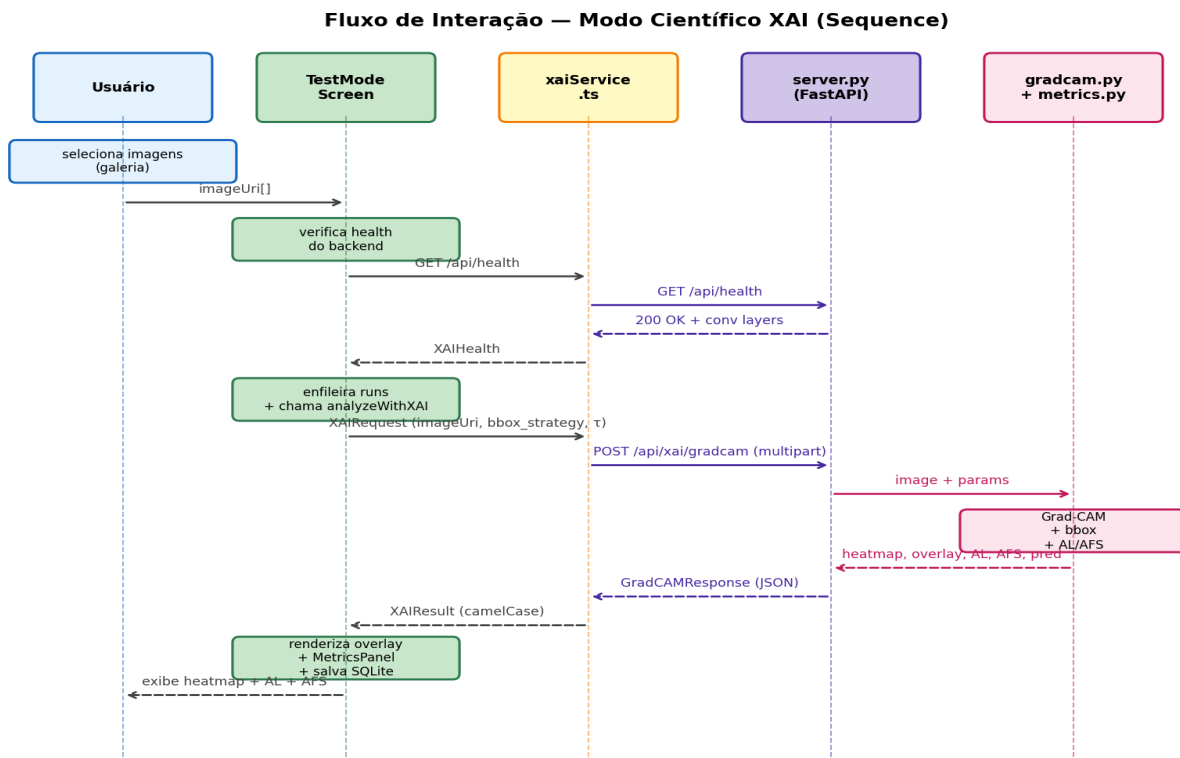


Figura 6. Diagrama de sequência do Modo Científico. O fluxo completo desde a seleção de imagens até a exibição do heatmap e das métricas AL/AFS, incluindo persistência local e possibilidade de exportação CSV.

6. Persistência e Exportação

Cada execução do Modo Científico é persistida localmente em xai_tests (SQLite). O esquema armazena os metadados necessários para análise posterior — inclusive o overlay PNG em base64 — dispensando re-execução do backend:

```
xai_tests: id · imageUrl · prediction · confidence  
al · afs · threshold  
bboxJson · bboxStrategy  
overlayB64 (PNG base64 – revisão offline)  
label (bem_enquadrada | mal_enquadrada | nao_classificada)  
createdAt
```

A exportação CSV é gerada por `utils/csvExport.ts` — uma linha por teste, com todas as colunas exceto o overlay (para manter o arquivo compacto). O arquivo é escrito no cache do app e aberto no share sheet do sistema operacional, permitindo análise imediata em planilhas ou Python/Pandas.

7. Stack Tecnológica

Componente	Tecnologia principal	Papel
App UI	React Native 0.81 + Expo 54	Runtime iOS/Android + APIs nativas
App ML (offline)	react-native-tflite + TF.js CPU	Inferência TFLite no dispositivo
App XAI client	Axios (xaiService.ts)	HTTP multipart → backend XAI
App Storage	expo-sqlite	analyses + xai_tests + CSV export
Backend API	FastAPI 0.110 + Uvicorn 0.29	Endpoint /api/xai/gradcam
Backend ML	TensorFlow 2.15 + tf-keras	Grad-CAM (GradientTape)
Backend CV	OpenCV 4.8 + NumPy	HSV mask, overlay JET, CLAHE
Deploy backend	Railway (Docker/Nixpacks)	Servidor público via HTTPS

Tabela 1. Principais tecnologias e seus papéis.